

not appear in the source document, then $\sum_{i:w_i=w} a_i^t$ is zero. The ability to produce OOV words is one of the primary advantages of pointer-generator models; by contrast models such as our baseline are restricted to their pre-set vocabulary.

The loss function is as described in equations (6) and (7), but with respect to our modified probability distribution $P(w)$ given in equation (9).

2.3 Coverage mechanism

Repetition is a common problem for sequence-to-sequence models (Tu et al., 2016; Mi et al., 2016; Sankaran et al., 2016; Suzuki and Nagata, 2016), and is especially pronounced when generating multi-sentence text (see Figure 1). We adapt the *coverage model* of Tu et al. (2016) to solve the problem. In our coverage model, we maintain a *coverage vector* c^t , which is the sum of attention distributions over all previous decoder timesteps:

$$c^t = \sum_{t'=0}^{t-1} a^{t'} \quad (10)$$

Intuitively, c^t is a (unnormalized) distribution over the source document words that represents the degree of coverage that those words have received from the attention mechanism so far. Note that c^0 is a zero vector, because on the first timestep, none of the source document has been covered.

The coverage vector is used as extra input to the attention mechanism, changing equation (1) to:

$$e_i^t = v^T \tanh(W_h h_i + W_s s_t + w_c c_i^t + b_{\text{attn}}) \quad (11)$$

where w_c is a learnable parameter vector of same length as v . This ensures that the attention mechanism’s current decision (choosing where to attend next) is informed by a reminder of its previous decisions (summarized in c^t). This should make it easier for the attention mechanism to avoid repeatedly attending to the same locations, and thus avoid generating repetitive text.

We find it necessary (see section 5) to additionally define a *coverage loss* to penalize repeatedly attending to the same locations:

$$\text{covloss}_t = \sum_i \min(a_i^t, c_i^t) \quad (12)$$

Note that the coverage loss is bounded; in particular $\text{covloss}_t \leq \sum_i a_i^t = 1$. Equation (12) differs from the coverage loss used in Machine Translation. In MT, we assume that there should be a roughly one-to-one translation ratio; accordingly the final coverage vector is penalized if it is more or less than 1.

Our loss function is more flexible: because summarization should not require uniform coverage, we only penalize the overlap between each attention distribution and the coverage so far – preventing repeated attention. Finally, the coverage loss, reweighted by some hyperparameter λ , is added to the primary loss function to yield a new composite loss function:

$$\text{loss}_t = -\log P(w_t^*) + \lambda \sum_i \min(a_i^t, c_i^t) \quad (13)$$

3 Related Work

Neural abstractive summarization. Rush et al. (2015) were the first to apply modern neural networks to abstractive text summarization, achieving state-of-the-art performance on DUC-2004 and Gigaword, two sentence-level summarization datasets. Their approach, which is centered on the attention mechanism, has been augmented with recurrent decoders (Chopra et al., 2016), Abstract Meaning Representations (Takase et al., 2016), hierarchical networks (Nallapati et al., 2016), variational autoencoders (Miao and Blunsom, 2016), and direct optimization of the performance metric (Ranzato et al., 2016), further improving performance on those datasets.

However, large-scale datasets for summarization of *longer* text are rare. Nallapati et al. (2016) adapted the DeepMind question-answering dataset (Hermann et al., 2015) for summarization, resulting in the *CNN/Daily Mail* dataset, and provided the first abstractive baselines. The same authors then published a neural *extractive* approach (Nallapati et al., 2017), which uses hierarchical RNNs to select sentences, and found that it significantly outperformed their abstractive result with respect to the ROUGE metric. To our knowledge, these are the only two published results on the full dataset.

Prior to modern neural methods, abstractive summarization received less attention than extractive summarization, but Jing (2000) explored cutting unimportant parts of sentences to create summaries, and Cheung and Penn (2014) explore sentence fusion using dependency trees.

Pointer-generator networks. The pointer network (Vinyals et al., 2015) is a sequence-to-sequence model that uses the soft attention distribution of Bahdanau et al. (2015) to produce an output sequence consisting of elements from

the input sequence. The pointer network has been used to create hybrid approaches for NMT (Gulcehre et al., 2016), language modeling (Merity et al., 2016), and summarization (Gu et al., 2016; Gulcehre et al., 2016; Miao and Blunsom, 2016; Nallapati et al., 2016; Zeng et al., 2016).

Our approach is close to the Forced-Attention Sentence Compression model of Miao and Blunsom (2016) and the CopyNet model of Gu et al. (2016), with some small differences: (i) We calculate an explicit switch probability p_{gen} , whereas Gu et al. induce competition through a shared softmax function. (ii) We recycle the attention distribution to serve as the copy distribution, but Gu et al. use two separate distributions. (iii) When a word appears multiple times in the source text, we sum probability mass from all corresponding parts of the attention distribution, whereas Miao and Blunsom do not. Our reasoning is that (i) calculating an explicit p_{gen} usefully enables us to raise or lower the probability of all generated words or all copy words at once, rather than individually, (ii) the two distributions serve such similar purposes that we find our simpler approach suffices, and (iii) we observe that the pointer mechanism often copies a word while attending to multiple occurrences of it in the source text.

Our approach is considerably different from that of Gulcehre et al. (2016) and Nallapati et al. (2016). Those works train their pointer components to activate only for out-of-vocabulary words or named entities (whereas we allow our model to freely learn when to use the pointer), and they do not mix the probabilities from the copy distribution and the vocabulary distribution. We believe the mixture approach described here is better for abstractive summarization – in section 6 we show that the copy mechanism is vital for accurately reproducing rare but in-vocabulary words, and in section 7.2 we observe that the mixture model enables the language model and copy mechanism to work together to perform abstractive copying.

Coverage. Originating from Statistical Machine Translation (Koehn, 2009), coverage was adapted for NMT by Tu et al. (2016) and Mi et al. (2016), who both use a GRU to update the coverage vector each step. We find that a simpler approach – summing the attention distributions to obtain the coverage vector – suffices. In this respect our approach is similar to Xu et al. (2015), who apply a coverage-like method to image cap-

tioning, and Chen et al. (2016), who also incorporate a coverage mechanism (which they call ‘distraction’) as described in equation (11) into neural summarization of longer text.

Temporal attention is a related technique that has been applied to NMT (Sankaran et al., 2016) and summarization (Nallapati et al., 2016). In this approach, each attention distribution is divided by the sum of the previous, which effectively dampens repeated attention. We tried this method but found it too destructive, distorting the signal from the attention mechanism and reducing performance. We hypothesize that an early intervention method such as coverage is preferable to a post hoc method such as temporal attention – it is better to *inform* the attention mechanism to help it make better decisions, than to *override* its decisions altogether. This theory is supported by the large boost that coverage gives our ROUGE scores (see Table 1), compared to the smaller boost given by temporal attention for the same task (Nallapati et al., 2016).

4 Dataset

We use the *CNN/Daily Mail* dataset (Hermann et al., 2015; Nallapati et al., 2016), which contains online news articles (781 tokens on average) paired with multi-sentence summaries (3.75 sentences or 56 tokens on average). We used scripts supplied by Nallapati et al. (2016) to obtain the same version of the data, which has 287,226 training pairs, 13,368 validation pairs and 11,490 test pairs. Both the dataset’s published results (Nallapati et al., 2016, 2017) use the *anonymized* version of the data, which has been pre-processed to replace each named entity, e.g., *The United Nations*, with its own unique identifier for the example pair, e.g., @entity5. By contrast, we operate directly on the original text (or *non-anonymized* version of the data),² which we believe is the favorable problem to solve because it requires no pre-processing.

5 Experiments

For all experiments, our model has 256-dimensional hidden states and 128-dimensional word embeddings. For the pointer-generator models, we use a vocabulary of 50k words for both source and target – note that due to the pointer network’s ability to handle OOV words, we can use

²at www.github.com/abisee/pointer-generator

	ROUGE			METEOR	
	1	2	L	exact match	+ stem/syn/para
abstractive model (Nallapati et al., 2016)*	35.46	13.30	32.65	-	-
seq-to-seq + attn baseline (150k vocab)	30.49	11.17	28.08	11.65	12.86
seq-to-seq + attn baseline (50k vocab)	31.33	11.81	28.83	12.03	13.20
pointer-generator	36.44	15.66	33.42	15.35	16.65
pointer-generator + coverage	39.53	17.28	36.38	17.32	18.72
lead-3 baseline (ours)	40.34	17.70	36.57	20.48	22.21
lead-3 baseline (Nallapati et al., 2017)*	39.2	15.7	35.5	-	-
extractive model (Nallapati et al., 2017)*	39.6	16.2	35.3	-	-

Table 1: ROUGE F_1 and METEOR scores on the test set. Models and baselines in the top half are abstractive, while those in the bottom half are extractive. Those marked with * were trained and evaluated on the anonymized dataset, and so are not strictly comparable to our results on the original text. All our ROUGE scores have a 95% confidence interval of at most ± 0.25 as reported by the official ROUGE script. The METEOR improvement from the 50k baseline to the pointer-generator model, and from the pointer-generator to the pointer-generator+coverage model, were both found to be statistically significant using an approximate randomization test with $p < 0.01$.

a smaller vocabulary size than Nallapati et al.’s (2016) 150k source and 60k target vocabularies. For the baseline model, we also try a larger vocabulary size of 150k.

Note that the pointer and the coverage mechanism introduce very few additional parameters to the network: for the models with vocabulary size 50k, the baseline model has 21,499,600 parameters, the pointer-generator adds 1153 extra parameters (w_{h^*} , w_s , w_x and b_{ptr} in equation 8), and coverage adds 512 extra parameters (w_c in equation 11).

Unlike Nallapati et al. (2016), we do not pre-train the word embeddings – they are learned from scratch during training. We train using AdaGrad (Duchi et al., 2011) with learning rate 0.15 and an initial accumulator value of 0.1. (This was found to work best of Stochastic Gradient Descent, Adadelta, Momentum, Adam and RMSProp). We use gradient clipping with a maximum gradient norm of 2, but do not use any form of regularization. We use loss on the validation set to implement early stopping.

During training and at test time we truncate the article to 400 tokens and limit the length of the summary to 100 tokens for training and 120 tokens at test time.³ This is done to expedite training and testing, but we also found that truncating the article can *raise* the performance of the model

(see section 7.1 for more details). For training, we found it efficient to start with highly-truncated sequences, then raise the maximum length once converged. We train on a single Tesla K40m GPU with a batch size of 16. At test time our summaries are produced using beam search with beam size 4.

We trained both our baseline models for about 600,000 iterations (33 epochs) – this is similar to the 35 epochs required by Nallapati et al.’s (2016) best model. Training took 4 days and 14 hours for the 50k vocabulary model, and 8 days 21 hours for the 150k vocabulary model. We found the pointer-generator model quicker to train, requiring less than 230,000 training iterations (12.8 epochs); a total of 3 days and 4 hours. In particular, the pointer-generator model makes much quicker progress in the early phases of training. To obtain our final coverage model, we added the coverage mechanism with coverage loss weighted to $\lambda = 1$ (as described in equation 13), and trained for a further 3000 iterations (about 2 hours). In this time the coverage loss converged to about 0.2, down from an initial value of about 0.5. We also tried a more aggressive value of $\lambda = 2$; this reduced coverage loss but increased the primary loss function, thus we did not use it.

We tried training the coverage model without the loss function, hoping that the attention mechanism may learn by itself not to attend repeatedly to the same locations, but we found this to be ineffective, with no discernible reduction in repetition. We also tried training with coverage from the first

³The upper limit of 120 is mostly invisible: the beam search algorithm is self-stopping and almost never reaches the 120th step.